ISA and Quantized Block0 Design

1. Scope

This document describes the current midterm-stage design for accelerating a W8A8 quantized DeiT-Tiny workload on RV64IMC.

The current implementation includes:

- symmetric W8A8 quantization
- bare-metal RV64IMC kernels
- `Xqmac8` packed INT8 multiply-accumulate instruction
- `Xrequant` fixed-point requantization instruction
- Spike functional implementation
- complete connected DeiT-Tiny Transformer Block 0 execution

The current work is focused on ISA design and Spike-level evaluation. CV-Wally RTL integration is deferred to the next stage.

---

2. Target Workload

The target model is DeiT-Tiny.

```
Embedding dimension : 192
Attention heads     : 3
Head dimension      : 64
Transformer blocks  : 12
Tokens              : 197
MLP hidden size     : 768
```

The current integrated implementation evaluates one complete Transformer block:

```
Input
  |
  v
LayerNorm 1
  |
  v
QKV Projection
  |
  +----------------+
  |                |
  v                |
QK^T               V
  |                |
  v                |
```

```
Integer Softmax    |
  |                |
  +-------+--------+
          |
          v
    Attention x V
          |
          v
 Output Projection
          |
          v
   Residual Add 1
          |
          v
    LayerNorm 2
          |
          v
         FC1
          |
          v
      GELU LUT
          |
          v
         FC2
          |
          v
   Residual Add 2
          |
          v
    Block0 Output
```

---

3. Quantization Design

3.1 Quantization Format

The implementation uses symmetric W8A8 quantization.

```
Activation : INT8
Weight     : INT8
Accumulator: INT32
Zero point : 0
```

Activation quantization is per-tensor symmetric.

Weight quantization is per-output-channel symmetric.

The representable INT8 range is:

$$[-127, 127]$$

The value `-128` is intentionally not used by the quantizer.

3.2 Activation Quantization

For an FP32 activation tensor x:

$$s_x = \frac{\max |x|}{127}$$

and:

$$x_q = \text{clip}\left(\text{round}\left(\frac{x}{s_x}\right), -127, 127\right)$$

3.3 Per-Output-Channel Weight Quantization

For output channel j:

$$s_{w,j} = \frac{\max_i |w_{j,i}|}{127}$$

and:

$$w_{q,j,i} = \text{clip}\left(\text{round}\left(\frac{w_{j,i}}{s_{w,j}}\right), -127, 127\right)$$

3.4 Bias Quantization

For output channel j, the accumulator scale is:

$$s_{acc,j} = s_x s_{w,j}$$

The FP32 bias is converted to INT32 using:

$$b_{q,j} = \text{round}\left(\frac{b_j}{s_x s_{w,j}}\right)$$

The integer GEMM therefore computes:

$$acc_j = \sum_i x_{q,i} w_{q,j,i} + b_{q,j}$$

3.5 Fixed-Point Requantization

For output activation scale s_y:

$$M_j = \frac{s_x s_{w,j}}{s_y}$$

The current implementation converts M_j to Q31:

$$M_{int,j} = \text{round}\left(M_j 2^{31}\right)$$

The requantized value is approximately:

$$y_q = \mathrm{sat}_{INT8}\left(\mathrm{round}\left(\frac{acc \times M_{int}}{2^{31}}\right)\right)$$

The rounding rule is:

```
round to nearest
ties away from zero
```

The final value is saturated to:

$$[-127, 127]$$

---

4. Baseline RV64IMC Execution

Without custom instructions, an INT8 dot product requires repeated scalar operations.

Conceptually:

```
load activation byte
load weight byte
multiply
accumulate
update activation pointer
update weight pointer
update loop counter
branch
```

For the complete Block0 baseline profile, the dominant instructions include:

```
addi    27.92%
lb      27.60%
bne     13.91%
addw    13.79%
mulw    13.79%
```

This motivates the packed MAC extension.

---

5. Xqmac8

5.1 Purpose

`Xqmac8` reduces scalar INT8 dot-product overhead by packing eight signed INT8 multiplications into one custom instruction.

It operates on two 64-bit source registers containing eight INT8 elements each.

5.2 Semantics

Conceptually:

$$rd \leftarrow int32\left(rd + \sum_{i=0}^{7} int8(rs1_i) \times int8(rs2_i)\right)$$

where:

```
rs1[7:0]    -> INT8 element 0
rs1[15:8]   -> INT8 element 1
...
rs1[63:56]  -> INT8 element 7

rs2[7:0]    -> INT8 element 0
...
rs2[63:56]  -> INT8 element 7
```

The eight products are accumulated into the existing INT32 accumulator stored in `rd`.

The result uses INT32 wrap semantics and is sign-extended to RV64.

5.3 Encoding

The current Spike implementation uses the RISC-V CUSTOM-0 opcode space.

```
opcode = 0x0B
funct3 = 000
funct7 = 0000000
```

Spike definitions:

```
MATCH_XQMAC8 = 0x0000000b
MASK_XQMAC8  = 0xfe00707f
```

5.4 Operand Behavior

Logical operands:

```
rd  : old INT32 accumulator and destination
rs1 : packed 8 x INT8 operand A
rs2 : packed 8 x INT8 operand B
```

This implies an important microarchitectural consideration:

```
rd is both a source and destination
```

Therefore the instruction has three logical read operands:

```
old rd
rs1
rs2
```

A future CV-Wally implementation must consider:

- register-file read-port requirements
- forwarding of the old `rd` accumulator
- dependency handling
- execution latency
- writeback behavior

This issue does not affect functional Spike execution but is important for RTL integration.

5.5 Expected Instruction Reduction

For a dot product of length K, scalar RV64IMC executes approximately K multiply operations.

With `Xqmac8`:

$$N_{Xqmac8} = \left\lceil \frac{K}{8} \right\rceil$$

when padding is used.

Examples:

```
K = 192 -> 24 Xqmac8 instructions
K = 768 -> 96 Xqmac8 instructions
K = 64  -> 8 Xqmac8 instructions
```

For Attention x V:

```
K = 197
```

the implementation pads to:

```
K = 200
```

which requires:

$$25$$

`Xqmac8` instructions per output element.

---

6. Xrequant

6.1 Purpose

After INT8 GEMM, the output is stored in an INT32 accumulator domain.

Scalar requantization requires several operations:

```
signed multiplication
sign handling
rounding
shift
saturation
```

`Xrequant` fuses the fixed-point conversion into one custom instruction.

6.2 Semantics

Inputs:

```
rs1 : signed INT32 accumulator
rs2 : signed INT32 Q31 multiplier
```

Conceptual operation:

```
product = int64(rs1) * int64(rs2)

magnitude = abs(product)

rounded =
```

```
    (magnitude + 2^30) >> 31

restore original sign

clamp to [-127, 127]

rd = sign-extended INT8 result
```

This implements:

```
round to nearest
ties away from zero
```

6.3 Encoding

```
opcode = 0x0B
funct3 = 001
funct7 = 0000000
```

Spike definitions:

```
MATCH_XREQUANT = 0x0000100b
MASK_XREQUANT  = 0xfe00707f
```

6.4 Scope of the Current Instruction

The current `Xrequant` is intentionally specialized.

It assumes:

```
symmetric quantization
zero point = 0
fixed Q31 shift = 31
```

It does not currently encode:

- arbitrary zero point
- arbitrary shift
- asymmetric output quantization

This reduces ISA complexity for the current workload.

A future design may generalize these parameters if workload analysis shows that the additional flexibility is useful.

---

7. Integer LayerNorm

The connected Block0 uses an integer LayerNorm implementation.

For a row with:

$$N = 192$$

the implementation first computes:

$$sum = \sum_i x_i$$

and:

$$sumsq = \sum_i x_i^2$$

The variance numerator is represented using integer arithmetic:

$$var_num = N \cdot sumsq - sum^2 + eps_term$$

An integer square root is then used to approximate the standard deviation.

The centered numerator is:

$$centered_i = Nx_i - sum$$

The normalized value is represented in fixed point before applying quantized affine parameters.

Current fixed-point formats include:

```
LayerNorm normalized value : Q15
LayerNorm affine terms     : Q20
```

The resulting output is requantized to INT8.

---

8. Integer Softmax

The current attention Softmax is implemented without floating point.

8.1 Input

The input is the INT32 result of QK^T.

The implementation performs:

```
row maximum subtraction
logit clipping
exponential LUT lookup
integer accumulation
probability normalization
```

8.2 Probability Format

The probability output uses unsigned-like non-negative INT8 values:

```
range : [0, 127]
scale : 1 / 127
```

8.3 Exponential LUT

A 256-entry lookup table is used for the exponential approximation.

The logits are clipped to a bounded negative range before lookup.

The current implementation uses a workload-specific integer Softmax and does not attempt to reproduce general floating-point Softmax behavior.

---

9. GELU

The MLP activation uses a lookup table.

The LUT maps all possible INT8 FC1 outputs:

```
-128 ... 127
```

to INT8 values in the FC2 input scale.

The LUT is generated using the FP32 GELU reference:

```
torch.nn.functional.gelu(..., approximate="none")
```

followed by quantization into the FC2 input scale.

This removes floating-point nonlinear computation from the bare-metal workload.

---

10. Residual Connections

Residual operands may use different activation scales.

Before addition, each operand is requantized into a common residual scale.

For two inputs:

$$x_q,\ y_q$$

with scales:

$$s_x,\ s_y$$

and output scale:

$$s_o$$

the integer residual operation uses:

$$x'_q = requant\left(x_q, \frac{s_x}{s_o}\right)$$

$$y'_q = requant\left(y_q, \frac{s_y}{s_o}\right)$$

and:

$$o_q = sat_{INT8}\left(x'_q + y'_q\right)$$

The current implementation uses `Xrequant` for these scale conversions in X2.

---

11. Complete Block0 Dataflow

The connected integer Block0 is:

```
X0 INT8
  |
  v
Integer LayerNorm1
  |
  v
QKV INT8 GEMM
  |
  +-------------------------------+
  |                               |
  v                               v
Q / K                          V INT8
  |
  v
QK^T INT32
  |
  v
Integer Softmax INT8 probability
  |
  +-------------------------------+
  |
```

```
  v
Attention x V
  |
  v
Projection
  |
  v
Residual Add 1
  |
  v
Integer LayerNorm2
  |
  v
FC1
  |
  v
GELU LUT
  |
  v
FC2
  |
  v
Residual Add 2
  |
  v
Final Block0 INT8
```

Tensor shapes:

Stage	Shape
X0	197 x 192
LN1	197 x 192
QKV	197 x 576
Q	3 x 197 x 64
K	3 x 197 x 64
V	3 x 197 x 64
QK^T	3 x 197 x 197
Softmax	3 x 197 x 197
AV	3 x 197 x 64
Projection	197 x 192
Residual1	197 x 192
LN2	197 x 192
FC1	197 x 768

Stage	Shape
FC2	197 x 192
Final	197 x 192

---

12. Functional Correctness Strategy

Correctness is verified at several levels.

12.1 ISA-Level Tests

```
Xqmac8 directed tests
Xrequant directed tests
Xqmac8 randomized tests
Xrequant randomized tests
```

12.2 Operator-Level Tests

```
FC1
FC2
QKV
QK^T
Softmax
Attention x V
Projection
MLP
```

12.3 Complete Block Verification

The complete Block0 integer result from Spike is compared against the Python integer reference.

The final tensor contains:

$$197 \times 192 = 37,824$$

INT8 elements.

All final outputs match bit-exactly.

---

13. Quantization Error vs Implementation Error

The project distinguishes two different error sources.

Quantization error

```
FP32 reference
      !=
Integer reference
```

This difference is expected.

Implementation error

```
Python integer reference
      ==
Spike integer result
```

This equality is required.

The connected Block0 currently achieves:

```
Final MAE     : 0.078367
Final Max Err : 0.438896
Final SQNR    : 19.63 dB
```

relative to FP32.

At the same time, Spike is bit-exact with the Python integer implementation.

---

14. Observed Post-X2 Bottlenecks

After applying `Xqmac8` and `Xrequant`, the dominant X2 instructions are:

```
addi      27.29%
ld        25.17%
bne       13.39%
.insn     12.93%
sext.w    12.86%
```

The custom instructions substantially reduce scalar multiplication, but expose new bottlenecks.

14.1 Data Movement

Loads still account for approximately:

$$26.46\%$$

of main instructions.

Potential future directions:

- fused load + MAC
- cache optimization
- software-managed scratchpad
- limited dual issue

14.2 Address Generation

`addi` becomes the largest individual instruction class.

Potential future direction:

- post-increment load/store

14.3 Loop Control

Branch instructions remain significant.

Potential future direction:

- hardware loop support

14.4 RV64 Sign Extension

`sext.w` accounts for a significant fraction of X2 execution.

This is currently treated as a compiler/custom-instruction interface issue associated with INT32 accumulator canonicalization on RV64.

It should be investigated before proposing an additional hardware instruction.

---

15. CV-Wally Integration Considerations

The next project stage will require implementing at least one representative kernel in CV-Wally.

Important design questions include:

Xqmac8

- How many execution cycles should one `Xqmac8` require?
- Is eight-way INT8 multiplication fully parallel?
- How is the old `rd` accumulator read?
- Are additional register-file ports required?
- Can forwarding supply the previous accumulator?
- Does the custom instruction affect the critical path?

Xrequant

- Is the signed 32 x 32 multiply implemented using the existing multiplier?
- How many cycles are required?

- Can rounding, shifting, and saturation be fused into the multiplier pipeline?
- Does the instruction require additional pipeline stages?

Memory system

The Spike profile indicates that load and address-generation overhead become important after custom compute optimization.

Future RTL evaluation should therefore measure:

- cycle count
- CPI
- stalls
- memory access behavior
- critical path
- synthesized area
- performance per area

---

16. Reproducibility

Spike base commit:

```
0bff121
```

Custom ISA patch:

```
patches/spike-xqmac-xrequant.patch
```

The patch includes:

```
riscv/encoding.h
riscv/riscv.mk.in
riscv/insns/xqmac8.h
riscv/insns/xrequant.h
```

The patch was validated using:

```
git apply --check
```

against a clean Spike worktree based on the specified base commit.

Current project checkpoint:

```
Tag:
midterm-block0-v1

Block0 experiment commit:
5ebd201

Spike patch commit:
f507d92
```

---

17. Current Design Summary

The current ISA design follows the observed workload behavior:

```
Baseline profiling
      |
      v
Scalar MAC identified as a major bottleneck
      |
      v
Xqmac8
      |
      v
Requantization overhead remains
      |
      v
Xrequant
      |
      v
Complete Block0 evaluation
      |
      v
New bottlenecks:
load + address generation + loop control + sign extension
```

The design therefore follows the intended architecture methodology:

```
measure
  ->
identify bottleneck
  ->
propose ISA mechanism
  ->
evaluate
  ->
identify next bottleneck
```